

Remote Sensing and GIS

Assignment 8

Kaushal Singh [CH22BTECH11018]

3. Georeferencing

This part of the assignment focused on georeferencing, which means assigning correct real-world coordinates to a scanned map or raster image. This process is important because an old map image by itself cannot be used for spatial analysis until it is aligned with a coordinate system. In this exercise, I worked with a historical map of Bangalore and aligned it with a modern basemap in QGIS.

3.1 Using Basemaps

I started by loading a basemap through the **QuickMapServices** plugin. To do this, I installed the contributed service pack and added the **OSM Standard** basemap. This layer provided a georeferenced modern reference map, which was necessary for identifying common locations between the historical image and present-day Bangalore.

Once the basemap was loaded, I confirmed that it was displayed in **EPSG:3857**, which is the standard web map coordinate system used by many online map services. This created the reference environment required for georeferencing the scanned image.

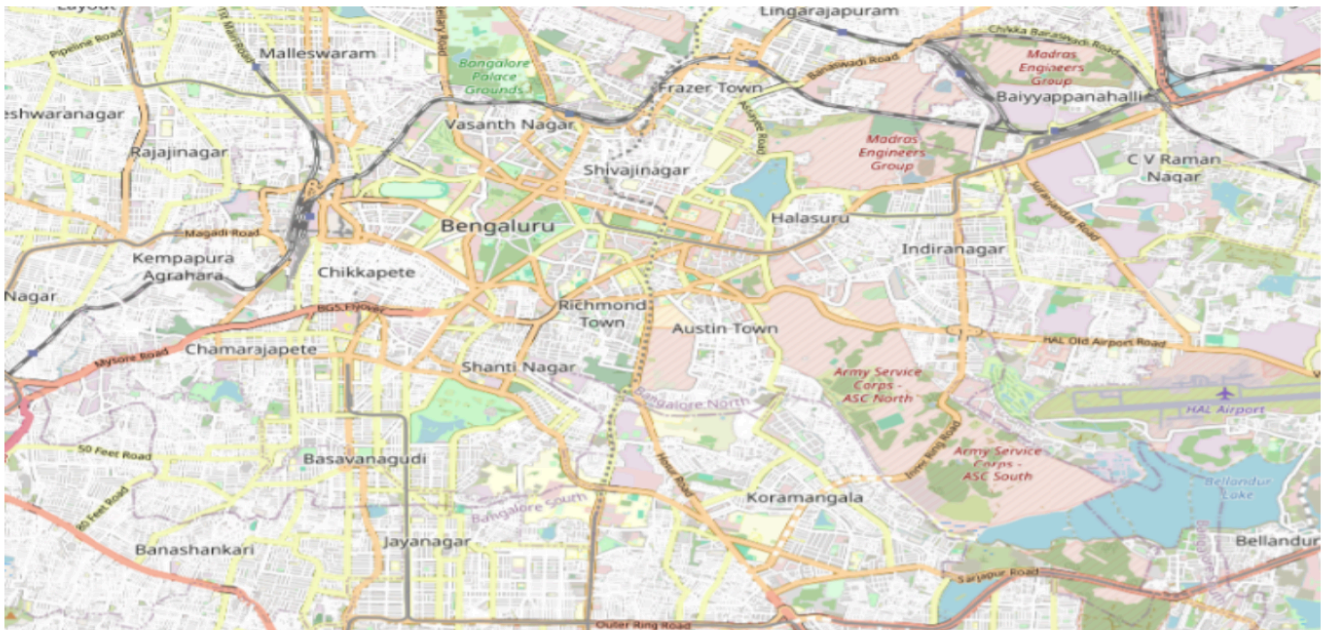


Challenge 3.1.1

Load the Dark Matter basemap by CartoDB. This is a minimalist basemap that renders OpenStreetMap data in a dark background.

To complete this challenge, I opened **QuickMapServices** and selected the **CartoDB Dark Matter** basemap. This added a dark-themed reference layer to the project.

The Dark Matter basemap is useful because its simple dark background makes other layers stand out more clearly. It is especially effective when bright symbols or labels need to be highlighted on top of the map.



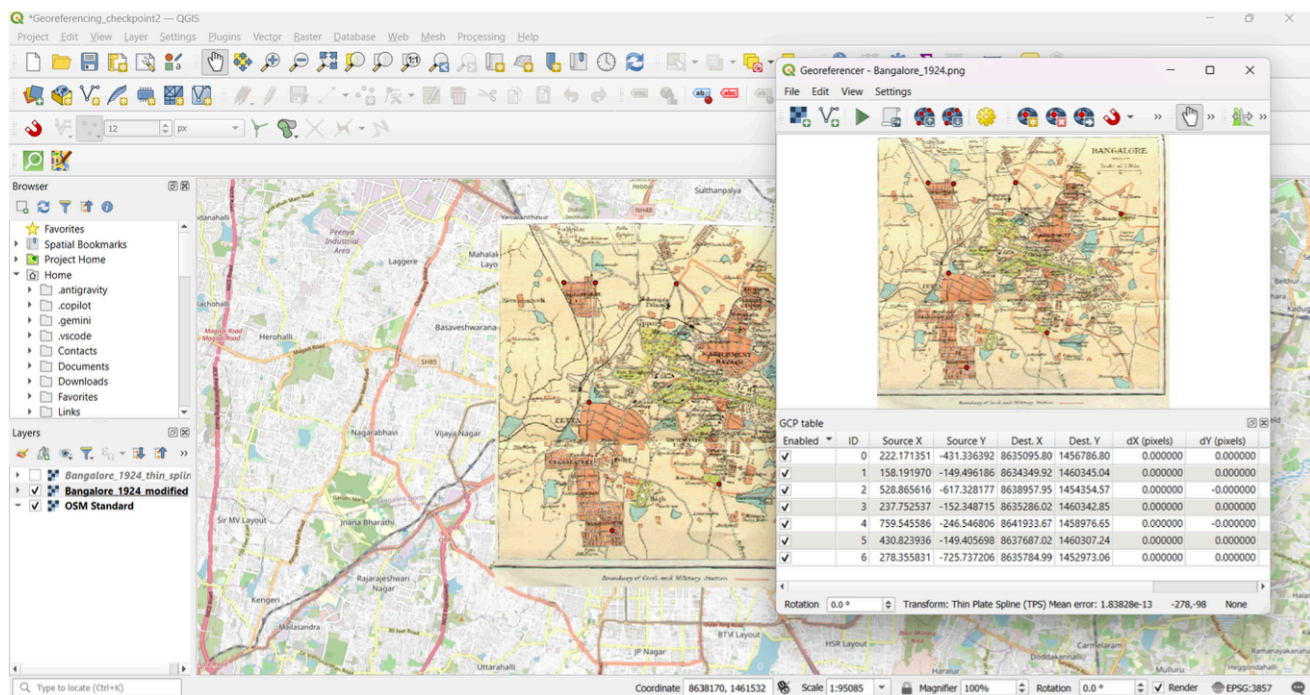
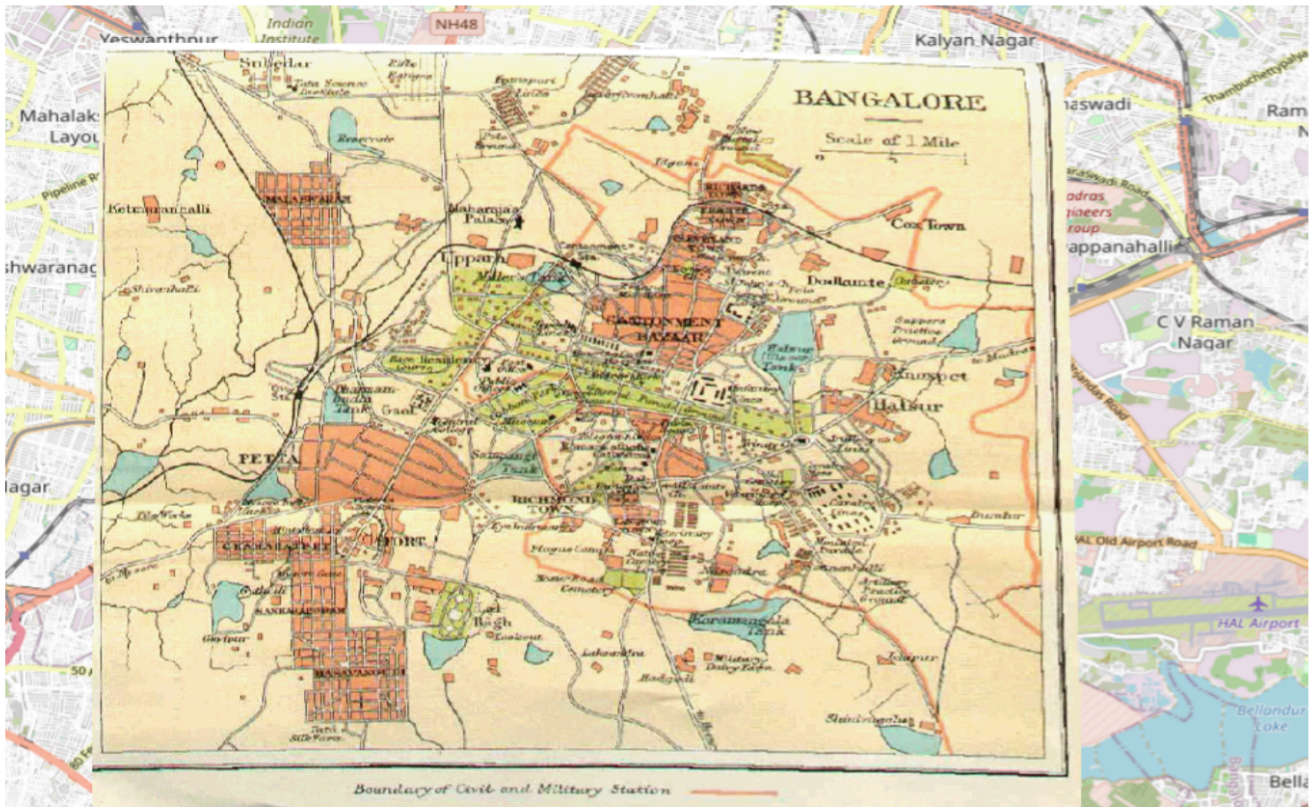
3.2 Using the Georeferencer

In this section, I used the **Georeferencer** tool in QGIS to align the scanned map with the basemap. I opened the raster image **Bangalore_1924.png** and identified visible locations that were common to both the old map and the modern basemap. These matching locations were added as **Ground Control Points (GCPs)**. For each GCP, I first clicked a location on the scanned image and then used the **From Map Canvas** option to pick the matching coordinate from the basemap.

I collected several such points, mainly at road intersections, corners, and other easily identifiable places. Since the transformation method being used was **Polynomial 2**, at least six GCPs were required. I also reviewed the GCP table to inspect the residual error values for each point.

If any point had a high error, I adjusted or removed it. When the point set became satisfactory, I opened the transformation settings and selected **Polynomial 2** as the transformation type, **Nearest Neighbour** as the resampling method, and **EPSG:3857** as the target CRS. The output was saved as **Bangalore_1924_modified.tif**, **LZW compression** was selected, and I enabled both **Save GCP points** and **Load in QGIS when done**.

Finally, I ran the georeferencing process. The transformed raster was loaded back into QGIS, where it aligned over the modern basemap. This confirmed that the old map had been successfully georeferenced.



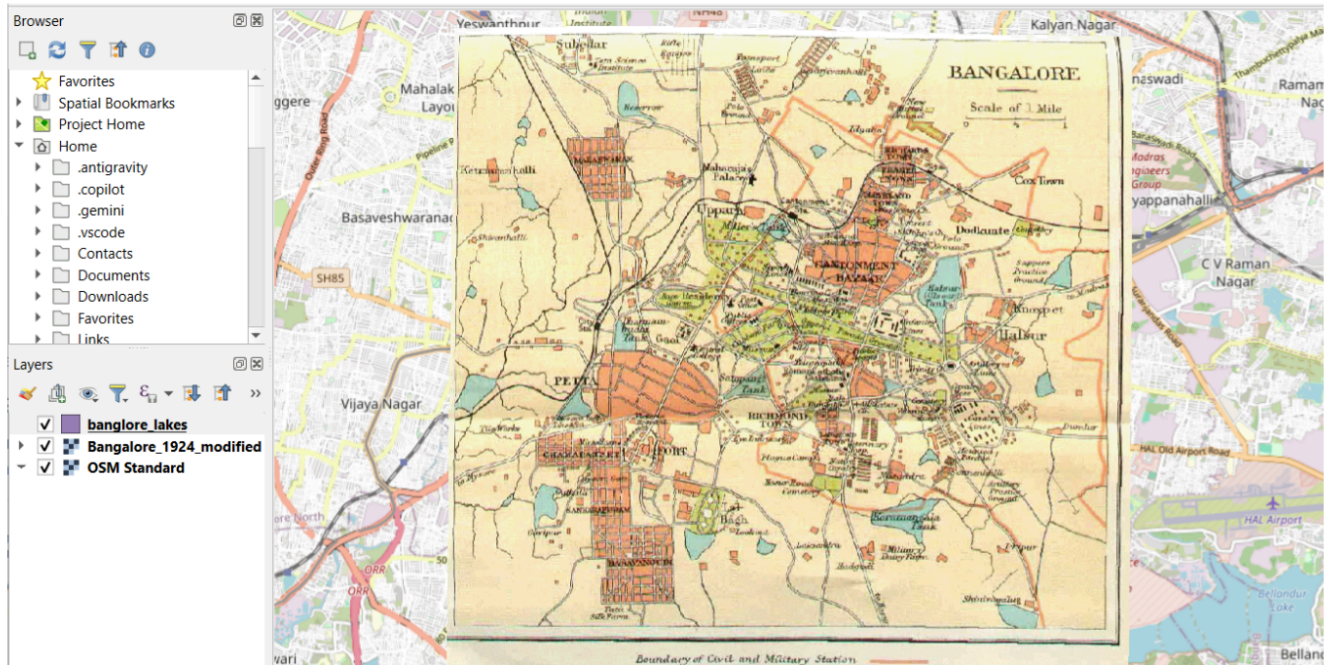
Challenge 3.2.1

In this exercise we used the Polynomial 2 technique. For datasets that require more aggressive transformation, you can use the Thin Plate Spline algorithm. This method is also known as Rubber Sheeting. Change the transformation setting to use Thin Plate Spline and run the georeferencer again. Compare the output with the previous result.

For this challenge, I changed the transformation method from **Polynomial 2** to **Thin Plate Spline** and ran the georeferencer again. Thin Plate Spline is more flexible because it can

stretch and bend the raster more strongly to fit the control points.

When I compared the output with the earlier result, I observed that the Thin Plate Spline version adjusted the map more aggressively in local areas. This can be helpful when the old map contains irregular distortions, although it may also introduce stronger warping in some parts of the image.



4. Data Editing

This section was about creating and editing vector data. Using the georeferenced historical Bangalore map, I digitized old lake boundaries and recorded their present condition. The aim was to create a clean polygon layer and attach useful attributes to each feature.

4.1 Attribute Forms

I opened the prepared QGIS project and created a new **GeoPackage** layer named **banglore_lakes**. The geometry type was set to **MultiPolygon** because the task involved digitizing lake boundaries as polygon features. I kept the default **EPSG:4326** coordinate system and then added two custom fields: **name** as text with a maximum length of **50**, and **status** as an integer field.

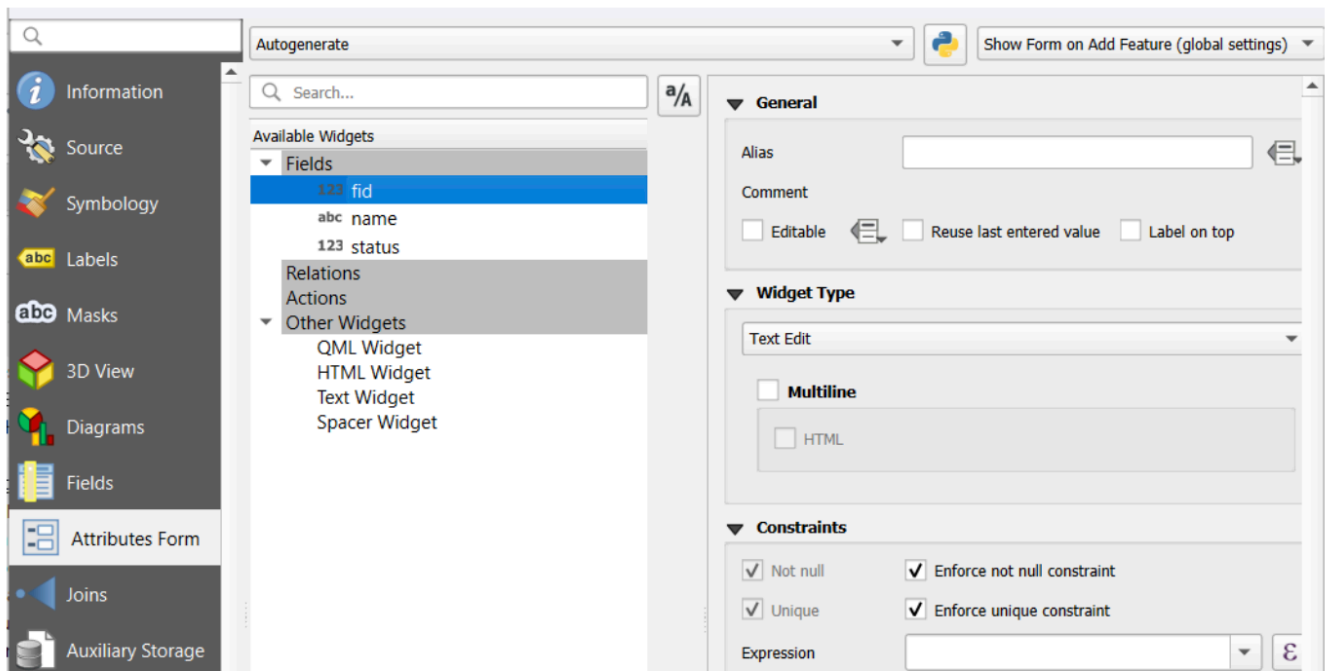
After creating the layer, I checked its attribute table to confirm that the fields had been added correctly. The **fid** field was already present because GeoPackage automatically maintains a unique ID for each feature.

Next, I configured the attribute form for the **status** field. I changed its widget type to **Value Map**, which allowed me to create a drop-down menu with fixed categories. I entered the

values as:

- 1 for **Healthy**
- 2 for **Partially Lost**
- 3 for **Lost**

I also made the **status** field mandatory by enabling **Not null** and **Enforce not null constraint**. This ensured that every digitized lake would have a proper status value.



Challenge 4.1.1

The **fid** column contains auto-increment unique id for each feature. The GeoPackage format requires this integer field to maintain data integrity. Manually overriding this id to a different value can cause data corruption. Edit the attribute form for the **fid** field so that it is not user-editable.

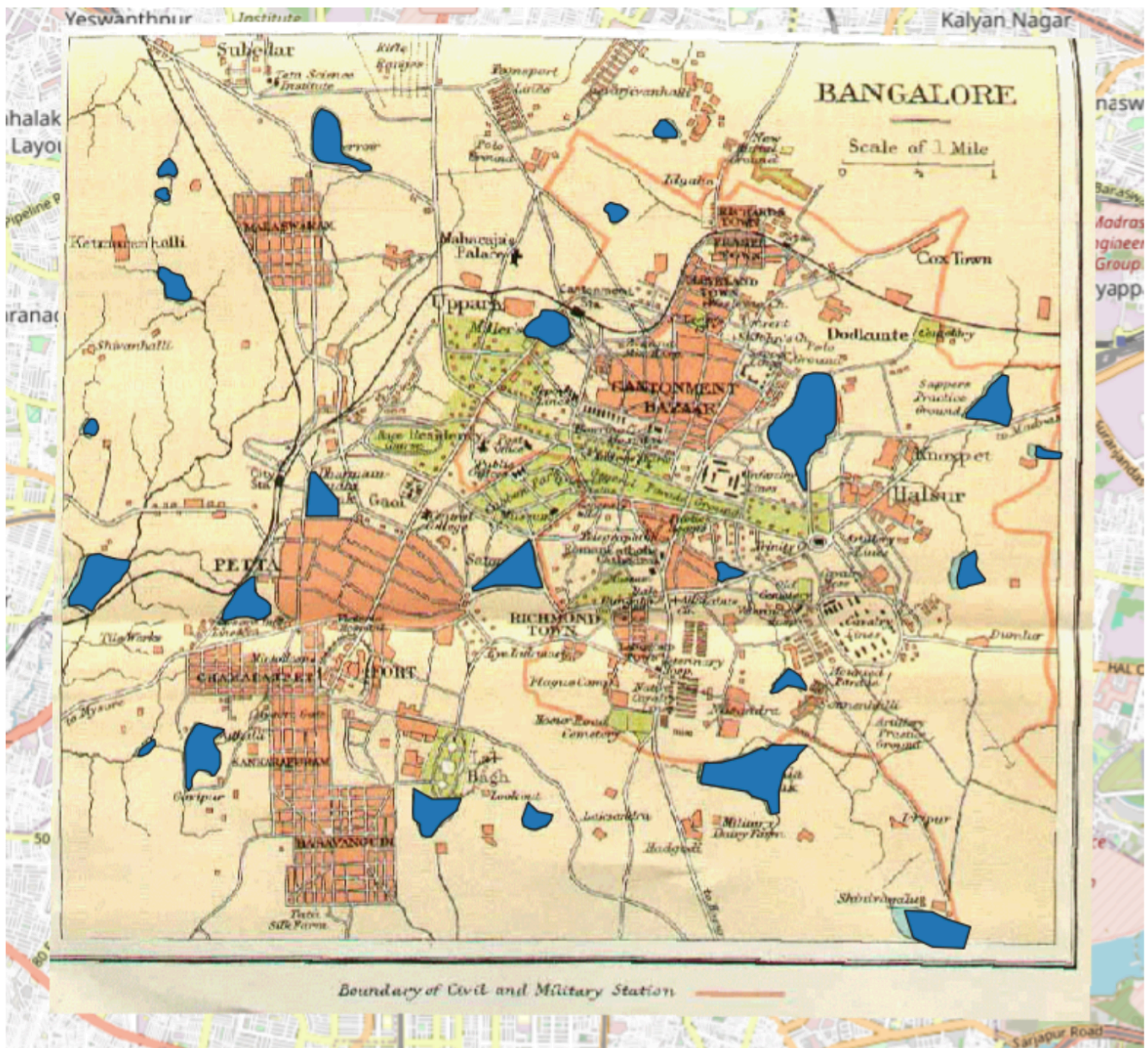
To complete this task, I opened the layer properties and edited the attribute form settings for the **fid** field. I changed the field configuration so that it would no longer be editable by the user in the form.

This is important because the **fid** value is automatically managed by GeoPackage. Preventing manual editing protects the dataset from accidental errors and helps maintain the integrity of the feature IDs.

4.2 Digitizing Polygons

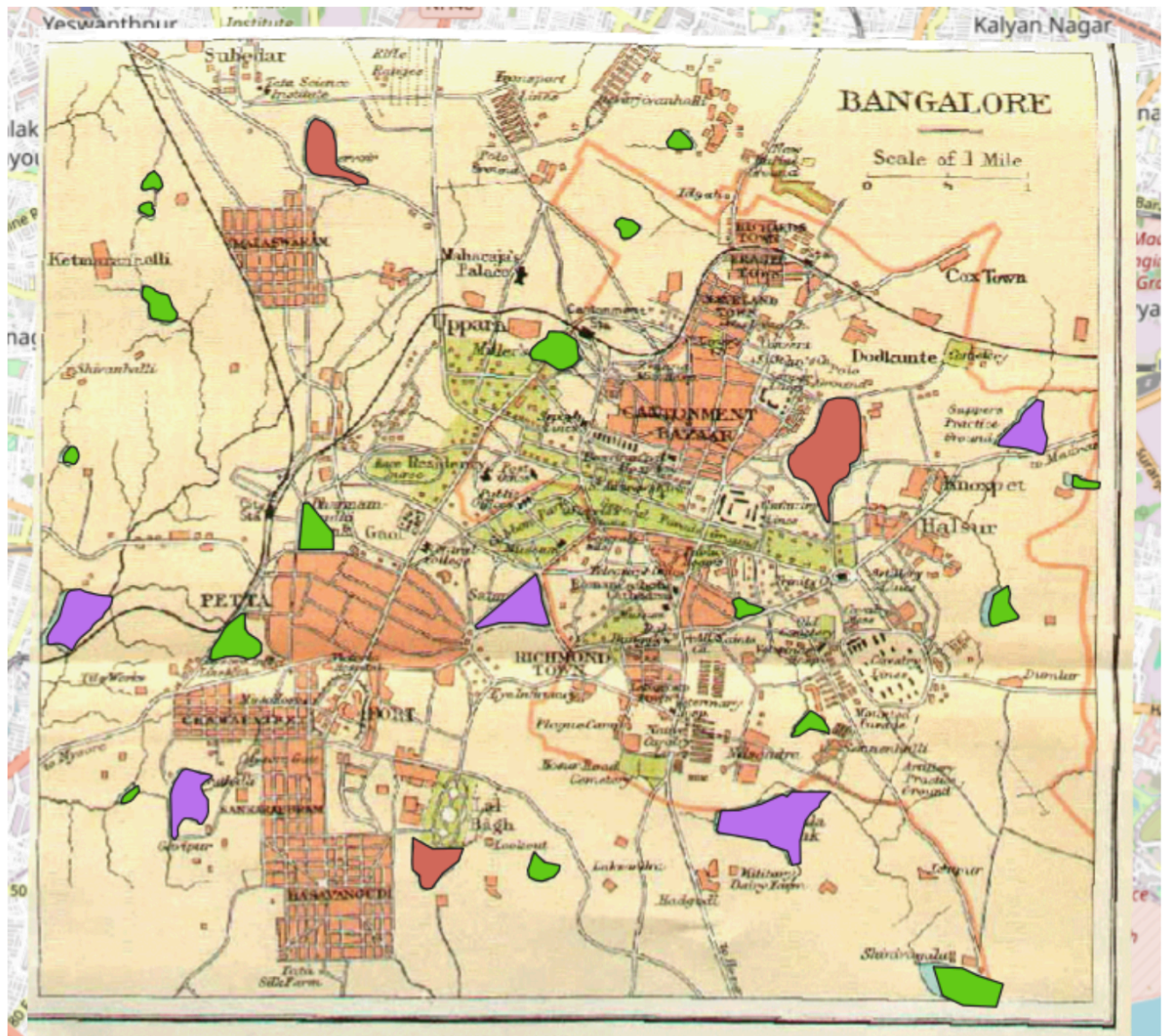
Before digitizing, I enabled the **Snapping Toolbar** so that vertices could be placed more accurately. This helps avoid geometry mistakes and makes polygon boundaries cleaner.

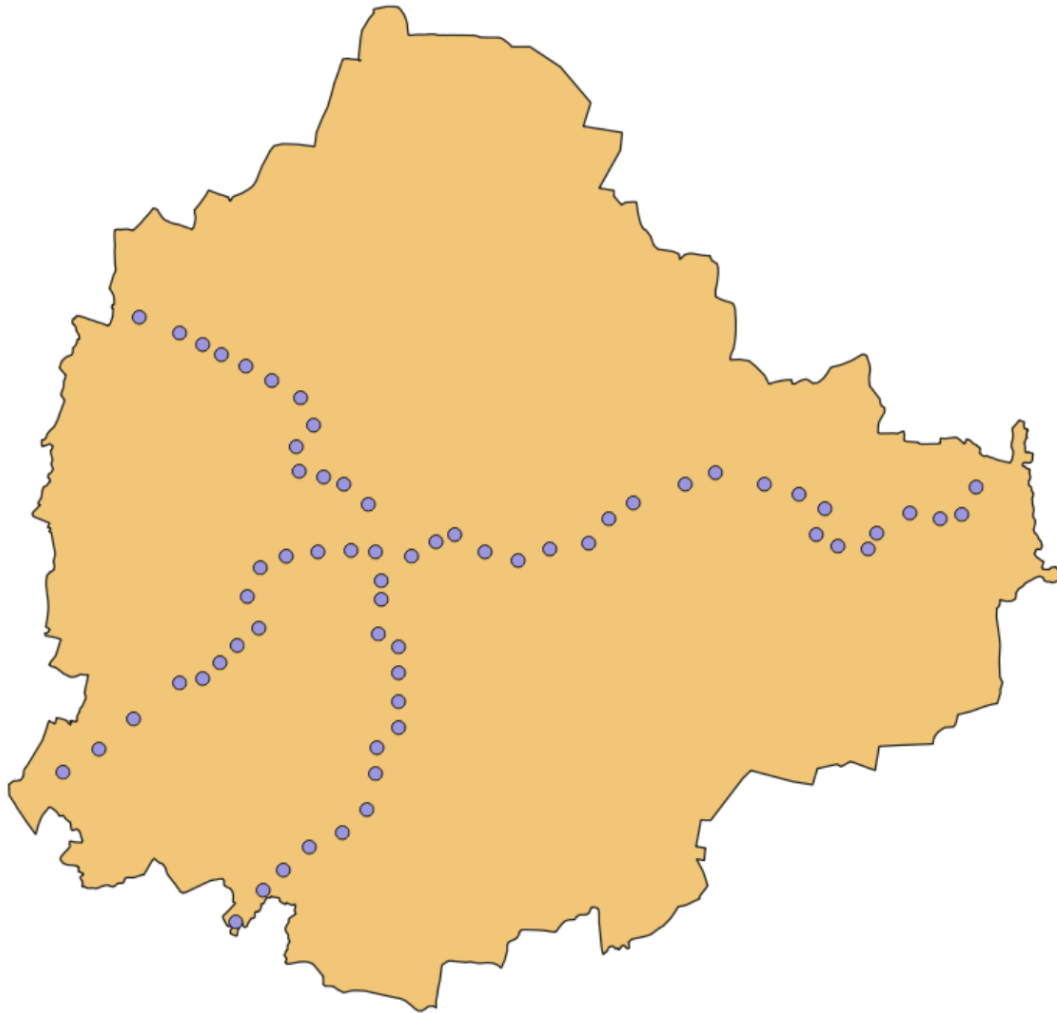
I then switched on editing for the **bangalore_lakes** layer and used the **Add Polygon Feature** tool to trace the lake boundaries visible in the historical raster. While digitizing, I compared the old map with the modern **OSM Standard** basemap so I could identify each waterbody and decide whether it was still present, partially lost, or completely lost.



After drawing each polygon, QGIS opened the attribute form. I entered the lake name wherever it was available and selected the correct **status** from the drop-down list. If the name was not visible, I left that field blank and only filled the status value.

I repeated this process for all the visible lakes and then saved the edits. When I checked the attribute table, the layer contained **24 digitized features**, each with an automatically assigned **fid** and the attributes I had entered during digitizing.





Challenge 4.2.1

Style the layer based on the status column. This column has categorical values that can be used assign a different color to each waterbody.

To style the lakes, I used the **Categorized** renderer in the layer styling panel and selected the **status** field as the classification column. This allowed QGIS to assign different colors to each category automatically.

As a result, the lakes were displayed in separate colors based on whether they were **Healthy**, **Partially Lost**, or **Lost**. This made the final map easier to interpret because the condition of each waterbody could be understood at a glance.

fid		osm_id
1	1	5760197744
2	2	5760197745
3	3	5760197749
4	4	5760197751
5	5	5760197754
6	6	5760197755
7	7	5760197757
8	8	5760197761
9	9	5760197762
10	10	5760197764
11	11	5760197765

5. Geoprocessing

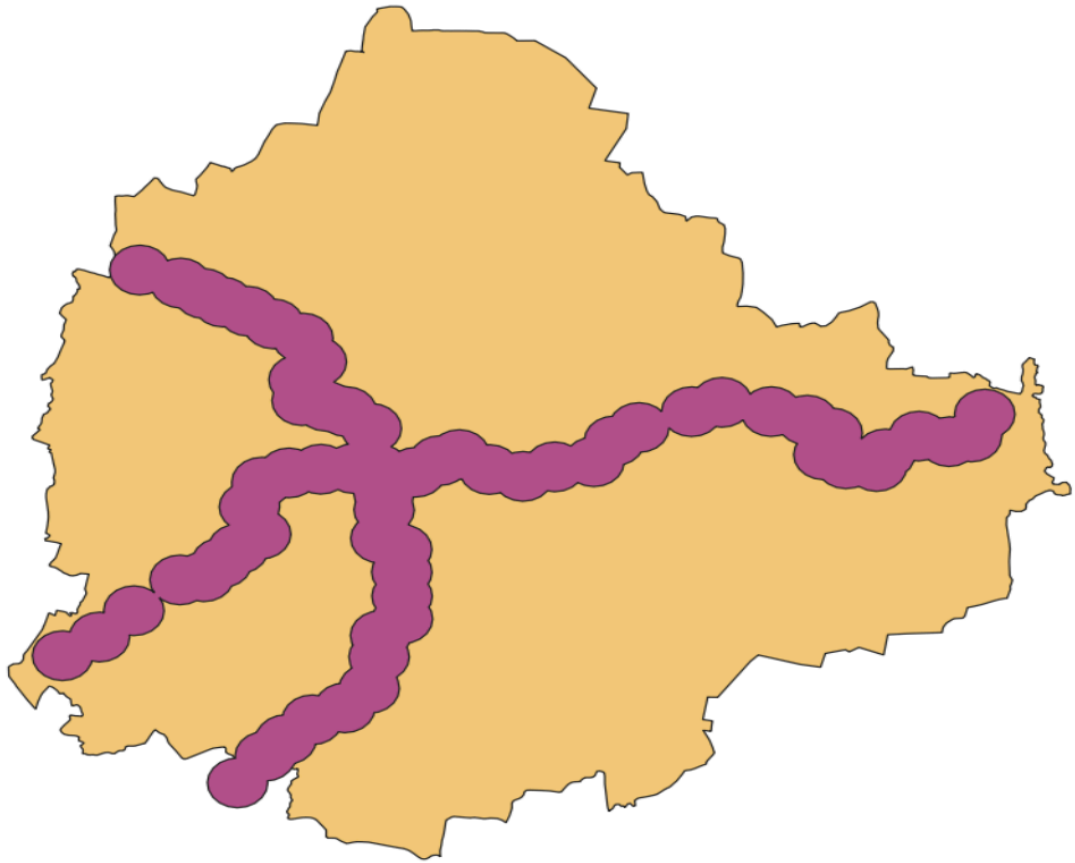
In this part of the assignment, I performed a spatial analysis workflow to estimate how many people in Bangalore live within 1 kilometer of a metro station. This required downloading station data, reprojecting layers, creating a buffer, and then using zonal statistics with a population raster.

5.1 Download OpenStreetMap Data

I first loaded the **bangalore.json** city boundary file into QGIS. After that, I used the **QuickOSM** plugin to query OpenStreetMap data for railway stations in Bangalore. I entered **railway** as the key and **station** as the value, while restricting the query to Bangalore. In the advanced options, I kept only **Node** and **Points** checked so the query returned point features.

The resulting layer included different kinds of railway stations, so I applied a filter to keep only those operated by **Bangalore Metro Rail Corporation Limited**. This left only the metro stations needed for the analysis.

Since the downloaded layer was temporary, I made it permanent and saved it as **railway_station_Bangalore.gpkg**. I also saved the project after this preprocessing step.



Challenge 5.1.1

You will notice that the attribute table for the `railway_station_bangalore` layer has many columns. Open the attribute table and delete all the columns except the `fid` and `osm_id` columns.

I opened the attribute table and used the **Delete Field** tool to remove the unnecessary columns. I kept only `fid` and `osm_id`, since these were enough to identify the features.

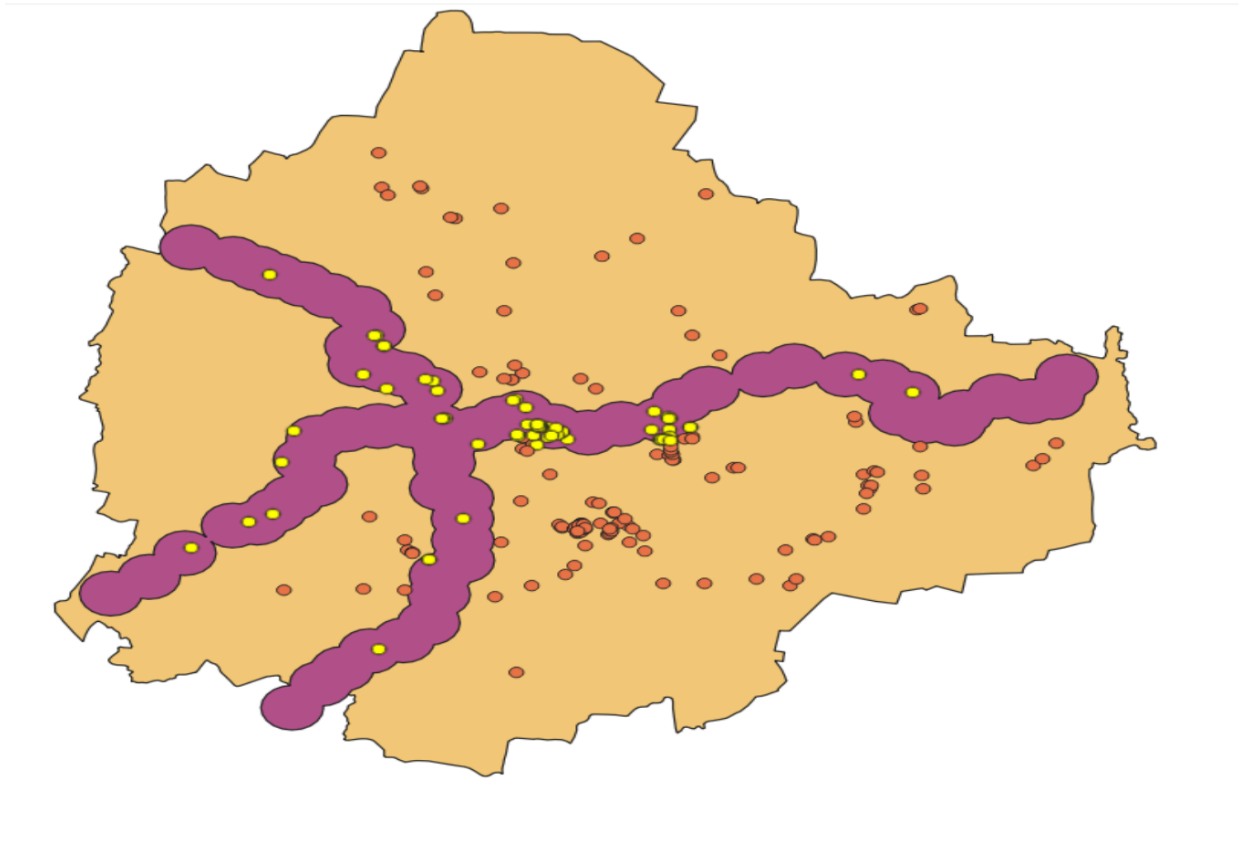
This cleanup made the layer simpler and easier to work with in the later steps of the analysis.

5.2 Reproject and Buffer

The metro station layer was originally in **EPSG:4326**, which uses degrees as units. Since the buffer distance had to be measured in kilometers, I first reprojected the station layer to **EPSG:32643 - WGS 84 / UTM Zone 43N**, which is more suitable for distance-based operations in Bangalore. The reprojected output was saved as **metro_stations_reprojected.gpkg**.

After reprojection, I applied the **Buffer** tool with a distance of **1 kilometer** and enabled the **Dissolve result** option so that all station buffers merged into a single polygon. This output was saved as **metro_stations_buffer.gpkg**, and it created a layer representing all areas within 1 km of a metro station.

To keep the project consistent with the other layers, I then reprojected the buffer layer back to **EPSG:4326** and saved it as **metro_station_buffer_reprojected.gpkg**. I removed the temporary intermediate layers so that only the final usable output remained in the project.



Challenge 5.2.1

Your data package contains a dataset called **bangalore_pubs.gpkg** with the location of all pubs within the city. Select all the pubs from the layer within 1km of a metro station.

To solve this challenge, I used the **Select by Location** tool from the Processing Toolbox. I selected the features from **bangalore_pubs.gpkg** that intersected the **metro_station_buffer_reprojected** layer.

This selected all pubs located within the 1 km metro influence zone. The output helped identify places that are spatially close to metro access.

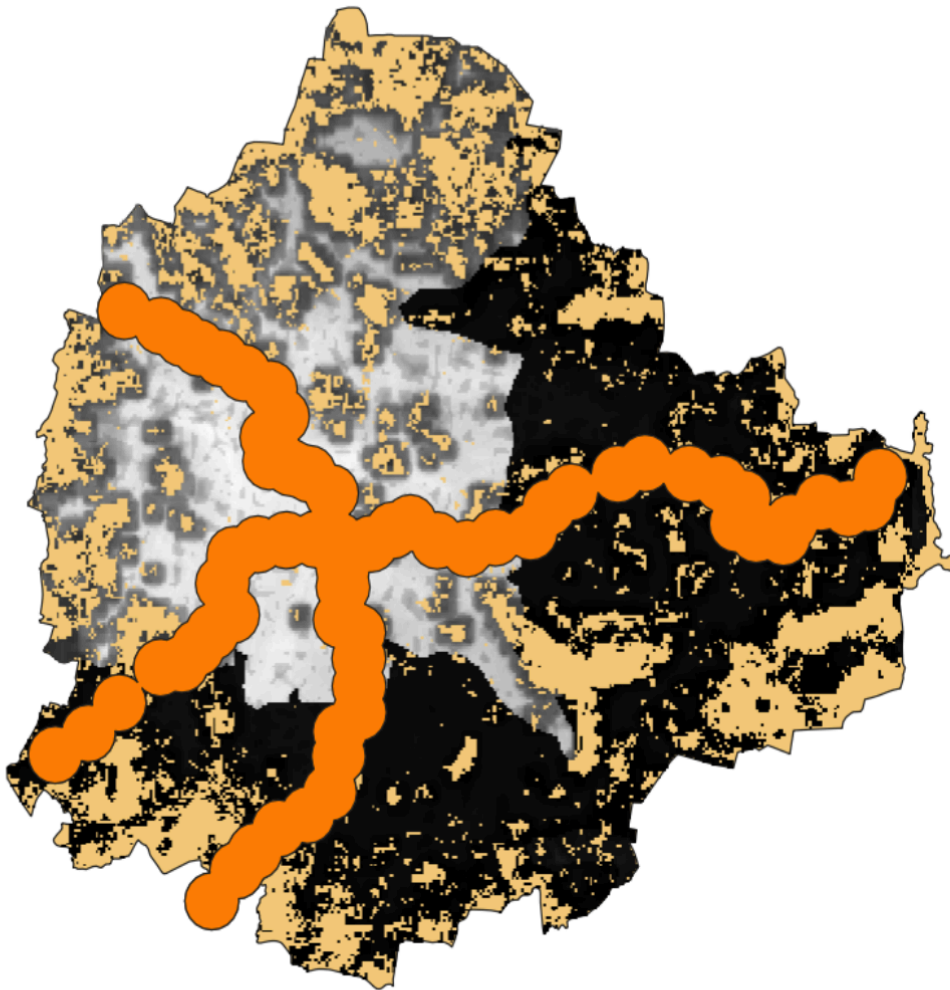
5.3 Calculate Zonal Statistics

In the final section, I added the raster **bangalore_ppp_2020_constrained.tif**, which contains gridded population values. Each pixel represents the estimated population for that

area, so the raster can be used to calculate how many people live inside a given polygon.

I used the **Zonal Statistics** tool with **metro_station_buffer_reprojected** as the input polygon layer and the population raster as the input raster. I selected only the **Sum** statistic, used **population_** as the output column prefix, and saved the output as **metro_station_buffer_pop.gpkg**.

When I opened the attribute table of the result, I saw a new field called **population_sum**. This field stored the estimated total population living within the 1 km metro station buffer zone.



Challenge 5.3.1

Repeat the Zonal Statistics operation on the **bangalore** layer to calculate the city's total population. Determine what percentage of the city population lives within 1km of a metro station.

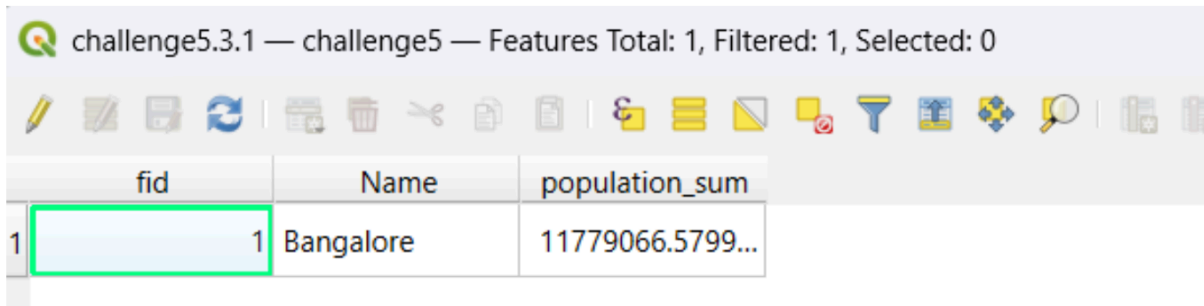
I repeated the **Zonal Statistics** process using the **bangalore** city boundary layer so that I could calculate the total population of the city. This produced another **population_sum** value representing the full population inside the Bangalore boundary.

After that, I compared the metro buffer population with the city total population using the formula:

$$(\text{population within 1 km of metro stations} / \text{total city population}) \times 100$$

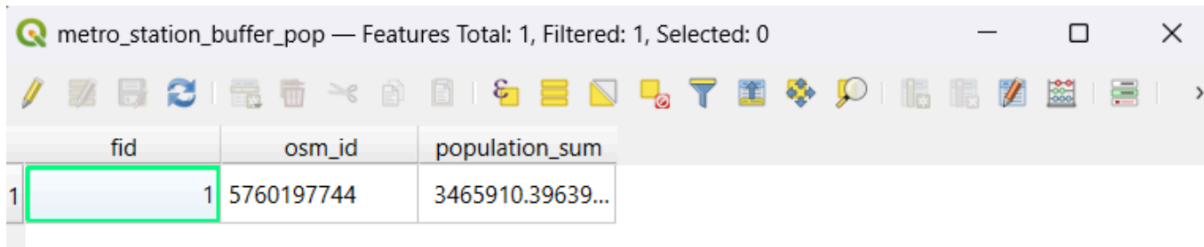
From the results, the metro buffer population was approximately **3,465,910**, while the total Bangalore population was approximately **11,779,067**. This means that about **29.42%** of the city's population lives within **1 km** of a metro station.

challenge5.3.1 — challenge5 — Features Total: 1, Filtered: 1, Selected: 0



fid	Name	population_sum
1	Bangalore	11779066.5799...

metro_station_buffer_pop — Features Total: 1, Filtered: 1, Selected: 0



fid	osm_id	population_sum
1	5760197744	3465910.39639...